

Especificación Técnica: Arquitectura y Flujos de Datos del Objeto GUMBO (v1.1)

1. El Paradigma GUMBO: De la Actualización Incremental al Estado Completo

El **GUMBO (Grand Unified Master Baseball Object)** representa la evolución crítica en la arquitectura de entrega de datos de MLBAM (Major League Baseball Advanced Media), sustituyendo los antiguos feeds fragmentados por un modelo de **estado completo**. Bajo esta arquitectura, cada objeto JSON generado es una instantánea total y autosuficiente del encuentro. Desde una perspectiva de ingeniería, GUMBO garantiza la **idempotencia**: procesar el mismo objeto múltiples veces no altera el estado del sistema receptor, lo que simplifica drásticamente la lógica de consumo al eliminar la necesidad de reconstrucciones manuales de secuencias de mensajes. Este cambio de paradigma permite una mayor flexibilidad de **schema-on-read** en entornos de Data Lake y bases de datos NoSQL, superando los formatos rígidos y altamente personalizados de los sistemas legados. Al adherirse a estándares JSON modernos (v1.1), GUMBO facilita la **hidratación de datos** en tiempo real, permitiendo que el "estado de la verdad" sea consistente y facilitando la ingesta masiva antes de pasar a los métodos específicos de adquisición.

2. Estrategias de Adquisición de Datos: Pull (Stats API) vs. Push (ActiveMQ)

La adquisición de datos en el ecosistema GUMBO es una decisión de arquitectura basada en los requisitos de latencia y volumen del sistema receptor. Los ingenieros deben elegir entre el sondeo activo o la escucha de eventos en vivo.

Atributo	Pull (Stats API)	Push (ActiveMQ)
Protocolo	HTTP (REST)	Mensajería (TCP/Websocket)
Latencia	~12 segundos	1 - 2 segundos
Caso de Uso	Desarrollo, auditoría y recuperación histórica	Producción en vivo y analítica en tiempo real
Endpoint / Topic	api/v1.1/game/{game_pk}/feed/live	mlbam.feed.2.0.game.state.lite.{venueId}

Para la recuperación de estados históricos o análisis forense, el sistema utiliza el parámetro **timecode** (formato `yyyymmdd_#####`). Es imperativo que el proceso de ingeniería recupere primero la lista de estados válidos desde el endpoint de `/timestamps` antes de intentar consultar un estado específico. Este mecanismo permite "viajar en el tiempo" a través del objeto GUMBO, manteniendo siempre la misma jerarquía de nodos estandarizada independientemente del transporte.

3. Jerarquía de Nodos Principales y Estructura del Documento JSON

La organización lógica de GUMBO se divide en tres nodos de nivel superior (`metaData`, `gameData`, `liveData`), diseñados para optimizar el filtrado y la indexación en sistemas de almacenamiento masivo.

3.1. Nodo `metaData`: Control de Estado y Sincronización

Este nodo actúa como el plano de control para la sincronización del feed.

- **wait:** Atributo numérico que define el intervalo de polling recomendado (en segundos) para evitar el estrangulamiento de la API.
- **timestamp:** Marca de tiempo que valida la última actualización del evento, esencial para detectar condiciones de carrera (*race conditions*) en sistemas concurrentes.
- **gameEvents** y **logicalEvents:** Arreglos que sirven como punteros externos hacia tipos de eventos normalizados, garantizando la integridad referencial del esquema.

3.2. Nodo gameData: Contexto y Entidades Estáticas del Encuentro

Contiene la infraestructura estática y las entidades del juego. El subnodo **players** utiliza una estructura de llave única de alta eficiencia compuesta por el prefijo ID + playerId (ej. ID429664), permitiendo un mapeo de entidades instantáneo para correlacionar con otros endpoints de person o stats. Un aspecto crítico para la automatización de alertas es la gestión de los *flags* en este nodo. Los indicadores de noHitter y perfectGame operan bajo una lógica de negocio estricta: **solo se activan tras cinco entradas completas (5 innings)** . Omitir esta condición en la lógica de alertas puede generar falsos positivos en las primeras fases del encuentro.

3.3. Nodo liveData: Dinámica de Juego y Telemetría en Tiempo Real

Es el núcleo dinámico que contiene la narrativa del juego. Dentro del subnodo **plays**, el arreglo **allPlays** es la estructura fundamental. El atributo **atBatIndex** sirve como clave primaria secuencial y **comienza estrictamente en cero** , un detalle técnico vital para evitar errores de *off-by-one* durante la reconstrucción secuencial de los turnos al bate. Además, el nodo **about** dentro de cada jugada incluye el **captivatingIndex** (escala 0–100). Este valor es una métrica de prioridad estratégica para sistemas de ingeniería que gestionan el almacenamiento en caché de "hot-data" o la priorización automática de *highlights* en tiempo real.

4. Deep Dive en Telemetría de Pícheo y Statcast

La integración de datos cinemáticos y espaciales transforma a GUMBO en una herramienta de analítica avanzada, marcando la transición definitiva de la era Pitch-f/x a las métricas mejoradas de Statcast disponibles desde 2015.

- **Atributos de pitchData:** Incluye parámetros de alta resolución como **startSpeed** (velocidad inicial a 50 pies) y **endSpeed** (al cruzar el plato). El movimiento se desglosa en **pfxX** y **pfxZ** (desplazamiento horizontal/vertical en pulgadas), junto con las coordenadas de liberación **x0**, **y0**, **z0**.
- **Atributos de hitData:** Captura métricas de contacto como **launchSpeed** (exit velocity), **launchAngle** y **totalDistance**.
- **Nota de Precisión de Ingeniería:** A diferencia de los datos de pícheo derivados de sensores, las coordenadas en **hitData.coordinates** son **ingresadas manualmente por un operador en el palco de prensa** . Los ingenieros deben considerar esta latencia humana y posible margen de error al procesar la posición de la bola puesta en juego.
- **Sistema de Strike Zone:** Las Plate Zones (1-14) se definen desde la **perspectiva del lanzador** . Por diseño, la Zona 1 es "alta y afuera" específicamente para un **bateador diestro** .

5. Integración en Sistemas de Análisis Profesional y Boxscore

La consolidación estadística en el nodo boxscore es la fuente definitiva para el análisis acumulativo y reportes post-partido.

- **Estructura de teamStats:** Desglosa el desempeño en grupos de batting, pitching y fielding. GUMBO aprovecha la **hidratación de datos** para incluir seasonStats dentro del perfil de cada jugador en el boxscore, proporcionando métricas de temporada actualizadas al instante sin requerir consultas externas.
- **Monitoreo de Hitos:** Los ingenieros pueden suscribirse a los indicadores de "All-Star Status" o los estados de juegos sin hits/perfectos para alimentar sistemas de notificación, siempre respetando el disparador de las 5 entradas mencionado anteriormente.

6. Conclusiones para la Implementación de Ingeniería

Para una implementación robusta de sistemas de datos MLB, los ingenieros deben adherirse a estas tres recomendaciones críticas:

- **Single Source of Truth:** GUMBO debe ser tratado como la única fuente de verdad para el estado del juego para evitar las inconsistencias de datos que ocurren al intentar fusionar múltiples fuentes de datos.
 - **Consumo de Statcast por Eras:** Los desarrolladores deben ser conscientes de la profundidad histórica de los datos para gestionar expectativas de analítica:
 - **1901–1968:** Solo nivel de Boxscore.
 - **1969–1988:** Nivel de Play-by-play (PBP).
 - **1989–2007:** Nivel de Pitch-by-pitch básico.
 - **2008–2014:** Pitch-by-pitch con datos cinemáticos de Pitch-f/x.
 - **2015–Presente:** Statcast completo con métricas de salida y distancia.
 - **Resiliencia y Reintentos:** Implementar estrategias de reintento con *backoff* exponencial para el acceso a datos históricos (especialmente pre-1980), donde se han documentado latencias mayores debido al particionamiento de bases de datos.
- Propiedad Intelectual:** Toda la información y descripción contenida en el feed GUMBO es propiedad de MLB Advanced Media, L.P. (MLBAM). Su uso está restringido a fines individuales y no comerciales bajo los términos de uso de la organización.